Информация о вебхуках

Общая информация о вебхуках

Вебхук – это механизм, при котором GitVerse отправляет HTTP-запрос на ваш URL при наступлении выбранных событий в репозитории.
Он нужен, чтобы связать действия в GitVerse с внешним сервисом: CI/CD, аналитикой, уведомлениями или вашей внутренней системой.
GitVerse позволяет задавать список событий, URL, тип контента и секрет для подписи запросов.

Какие события поддерживаются

При создании вебхука можно указать события, на которые он будет реагировать. В документации GitVerse перечислены, в частности: create, delete, fork, push, pull_request, pull_request_comment, pull_request_review.
По умолчанию, если событие не указано, используется push.
Это значит, что можно настроить вебхук как на простую отправку уведомления при каждом пуше, так и на более сложный сценарий с PR-ревью.

Создание вебхука

Необходимо перейти в настройки репозитория и перейти во вкладку “вебхуки”.

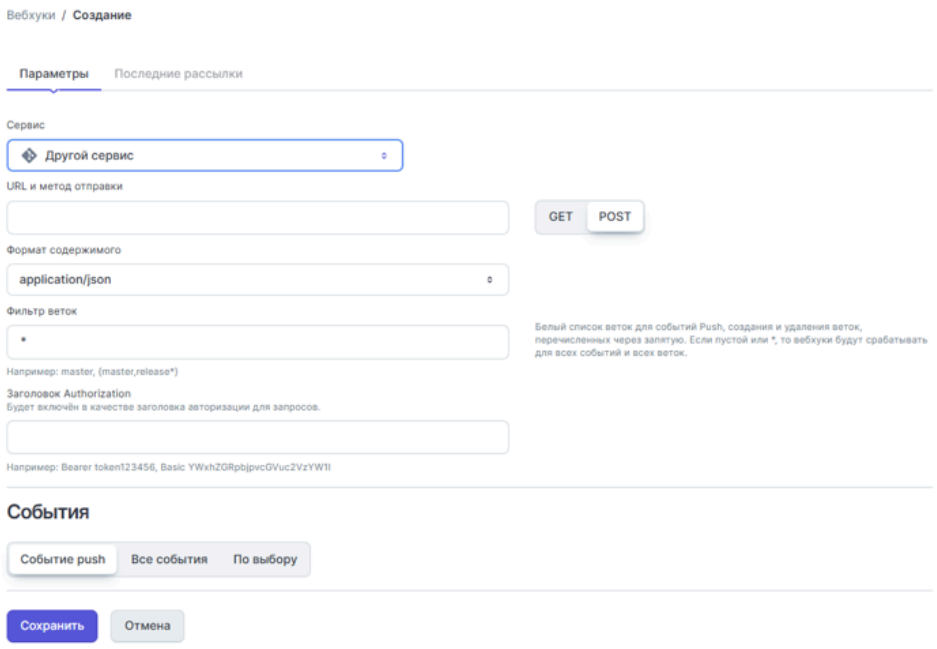
Для создания используется POST /repos/{owner}/{repo}/hooks.
В теле запроса передают JSON с полями active, events и config.
В config обязательно указывают url; дополнительно можно задать content_type, secret и http_method.
Для работы нужен токен с правами **Репозитории → Запись**.

Пример структуры запроса:

```
{
  "active": true,
  "events": ["push", "pull_request"],
  "config": {
    "url": "https://example.com/webhook",
    "content_type": "json",
    "secret": "secret_key"
```

```
  }
}
```

Можно выбрать сервис, указать метод отправки, выбрать формат содержимого, фильтр веток и заголовков авторизации.



Страница создания вебхуков

URL (Uniform Resource Locator) – это полный адрес вашего сервера или API-эндпоинта, который готов принять входящий запрос.

- Обязательно используйте https:// для обеспечения безопасности передачи данных. Использование http:// крайне не рекомендуется, так как данные (включая токены авторизации) будут передаваться в открытом виде.
- Укажите доменное имя (например, api.mysite.com) или публичный IP-адрес сервера.
- Укажите конкретный путь к обработчику (например, /webhooks/receiver).

Пример корректного URL

https://api.yourdomain.com/v1/webhook-handler

Формат метода отправки

Метод определяет, каким образом сервер будет передавать информацию на ваш URL. В интерфейсе на скриншоте доступны два варианта:

1. **Метод GET**

- Данные передаются непосредственно в строке URL в виде параметров запроса (Query Parameters).
- https://api.mysite.com/webhook?event=push&id=123
- Этот метод небезопасен для передачи больших объемов данных и не позволяет передавать тело запроса (Body).

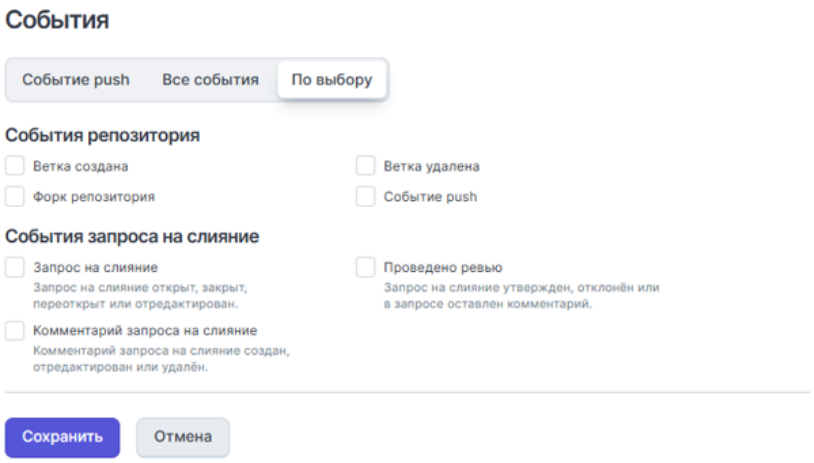
2. **Метод POST (Рекомендуемый)**

- Данные передаются в «теле» (Body) HTTP-запроса. Это стандарт для вебхуков.
- URL остается чистым (https://api.mysite.com/webhook), а вся полезная нагрузка (событие, ветка, данные пользователя) упаковывается в структуру, которая в вашем случае должна соответствовать формату application/json (как указано в поле «Формат содержимого» на скриншоте).

Преимущества:

- Позволяет передавать сложные и объемные JSON-структуры.
- Более высокий уровень безопасности.
- Соответствует стандартам современных API.

Также можно провести настройку событий.



Настройка события

GitVerse возвращает созданный вебхук с id, списком событий, конфигурацией, датами создания и обновления, а также статусом последнего ответа.

Проверка и просмотр

После создания можно посмотреть все вебхуки репозитория через GET /repos/{owner}/{repo}/hooks.
Если нужен один конкретный вебхук, используйте GET

/repos/{owner}/{repo}/hooks/{hook_id}.
Это полезно для диагностики: можно проверить, активен ли вебхук, какие события у него включены и какой был последний ответ от внешнего сервера.

Обновление вебхука

Если нужно изменить URL, события или секрет, используется PATCH /repos/{owner}/{repo}/hooks/{hook_id}.
Этот шаг нужен, когда меняется endpoint вашего сервера или вы хотите добавить новые типы событий без пересоздания вебхука.
На практике это удобно для поддержки интеграций, которые развиваются по мере использования.

Удаление вебхука

Когда вебхук больше не нужен, его удаляют через DELETE /repos/{owner}/{repo}/hooks/{hook_id}.
Это важно для безопасности и чистоты конфигурации, особенно если endpoint больше не обслуживается.
Удаление полностью убирает связь между репозиторием и внешним сервисом.

Как это работает на практике

1. Вы создаете endpoint на своем сервере, который принимает POST-запросы.
2. Регистрируете вебхук в GitVerse для нужного репозитория.
3. Выбираете события, например push и pull_request.
4. При наступлении события GitVerse отправляет запрос на ваш URL.
5. Ваш сервер принимает payload, проверяет подпись по secret и обрабатывает данные.
6. При необходимости обновляете или удаляете вебхук через API.

На что обратить внимание

- URL должен быть доступен извне и корректно принимать входящие запросы.
- Секретный ключ стоит обязательно использовать для проверки подлинности запросов.
- Для работы через API нужен токен с нужными правами.
- Если событие не поддерживается, вебхук не сработает, поэтому список событий надо задавать из допустимых значений.
- Если сервер отвечает ошибкой, это можно увидеть в last_response_status.

Короткая схема

1. Подготовить HTTP endpoint.
2. Создать вебхук через POST /repos/{owner}/{repo}/hooks.
3. Указать url, events, secret.
4. Проверить список через GET /repos/{owner}/{repo}/hooks.
5. При необходимости посмотреть один вебхук через GET /repos/{owner}/{repo}/hooks/{hook_id}.
6. Обновить через PATCH.
7. Удалить через DELETE.

Примеры сценариев автоматизации с вебхуками GitVerse

CI/CD и автопроверка

Самый частый сценарий – запуск сборки и тестов после push или pull_request.
Так можно автоматически проверять задания студентов, запускать линтеры и собирать артефакты без ручного участия.
Если проверка упала, система может сразу отправить уведомление в Telegram, Discord или другой канал.

Уведомления преподавателю

Вебхук можно использовать, чтобы сразу сообщать преподавателю о ключевых событиях: отправка решения, новый pull request, изменение статуса проверки.
Это особенно полезно в учебных модулях, где важно быстро видеть, кто сдал работу и у кого возникла ошибка.
Обычно такой сценарий реализуют через промежуточный сервис, который принимает webhook и пересылает сообщение дальше.

Отслеживание решений кандидатов

Если GitVerse используется как платформа для отбора, вебхуки можно применять для фиксации факта, что кандидат запушил код или открыл PR.
На основе этих событий можно строить таймлайн прохождения задания, считать количество попыток и время до успешной сдачи.
Это удобно для аналитики и сравнения кандидатов по реальным действиям в репозитории.

Автоматизация ревью

События pull_request и pull_request_review подходят для запуска внешнего AI-ревью или набора собственных проверок качества кода.
Например, после создания PR вебхук может отправить payload в сервис, который анализирует дифф, проверяет стиль и возвращает комментарии.
Такой подход помогает ускорить предварительный код-ревью до участия преподавателя или наставника.

Синхронизация с внешними системами

Вебхуки удобно использовать для интеграции GitVerse с LMS, CRM, Jira, Slack, Telegram и внутренними базами данных.
Например, при push можно автоматически обновлять статус задания, а при pull_request – создавать карточку в таск-трекере.
Это превращает GitVerse в источник событий, а не просто в хранилище кода.

Безопасность и контроль

Для надежности webhook-запросы стоит проверять по секрету и логировать входящие события.
Это важно, если по webhook запускаются автоматические действия, например деплой или оценивание.
Если событие пришло с ошибкой или дублируется, ваш сервис должен уметь его безопасно игнорировать.

Практические примеры

- push → запустить CI и проверить задание.
- pull_request → отправить PR на AI-ревью и уведомить преподавателя.
- pull_request_review → обновить статус проверки и записать замечания в БД.
- fork → зафиксировать начало работы кандидата.
- delete или create → обновить журнал активности репозитория.